Azure Recommended Practices Policy

Distilled from Microsoft Azure Well-Architected Framework, Cloud Adoption Framework and Azure Architecture Center guidance. Requirements use SHALL / MUST / SHOULD to state obligations; each chapter lists its sources.

Chapter 1 — Reliability and Resiliency

Intent

Workloads fail in ways that cannot be fully predicted, so reliability must be designed in from the start rather than added after an outage. This chapter requires that availability and recovery targets are derived from business needs, that redundancy and self-healing are built into critical flows, and that recovery is rehearsed before it is needed. Meeting these obligations keeps outages short, data loss bounded, and incident response predictable.

Requirements

Reliability targets

- Every production workload SHALL define an availability SLO, an RTO, and an RPO that are agreed in writing with business stakeholders. Targets set without business agreement cannot be used to justify design cost or to judge whether the design is adequate.
- The availability SLO of a production workload MUST reflect the quality promised to its users for the whole workload, including its dependencies, and MUST NOT be computed solely as the composite of dependency SLAs.
- Every production workload SHALL define at least one SLI for each availability SLO so that compliance with the SLO can be measured continuously.
- Reliability targets for a production workload SHOULD be reviewed at least annually and after every major incident, and the review outcome MUST be recorded.

Flow criticality and failure analysis

- Every production workload SHALL identify its user flows and system flows and rate each flow on a documented criticality scale. The ratings drive design investment, test priority, and incident prioritization.
- Every critical flow MUST have its own reliability targets, which MAY be stricter than the targets of the workload as a whole.
- Every production workload SHALL maintain a failure mode analysis that lists each component and dependency, its possible failure modes, the expected impact, and the planned mitigation.
- The failure mode analysis for a production workload MUST be updated whenever a new dependency is added or an architecture change alters a critical flow.

Redundancy and zone resilience

- Every production workload that has a critical flow SHALL deploy the components serving that flow redundantly, with no single instance acting as a single point of failure.
- Production workloads deployed to an Azure region that offers availability zones SHOULD use zone-redundant or zonal-spread deployments for compute, data, and networking components serving critical flows.

- Every Azure service tier chosen for a production workload MUST have a published SLA whose figure is at least equal to the availability SLO that the component must support.
- Every production workload with a regional-outage RTO shorter than the time needed to rebuild it SHALL be deployed to at least two Azure regions.
- Every stateful data store in a production workload SHALL use replication that matches the workload RPO, and the replication mode (synchronous or asynchronous) MUST be documented.

Scaling

- Every production workload SHALL define a scaling strategy covering the application, data, and infrastructure tiers, based on measured or forecast usage patterns.
- Scaling for production workloads SHOULD be automated through autoscale rules, and manual intervention MUST NOT be required to handle expected load peaks.
- Every autoscale configuration SHALL define both scale-out and scale-in rules together with explicit minimum and maximum instance counts.

Self-preservation and self-healing

- Every network call from a production workload to a remote dependency SHALL use a timeout, and calls to transient-fault-prone dependencies MUST use retries with exponential back-off and jitter. Unbounded waits and immediate retries amplify outages.
- Every production workload that depends on a remote service SHOULD apply a circuit breaker so that repeated failures stop further calls until the dependency recovers.
- Every critical flow in a production workload SHOULD define graceful degradation behavior that preserves core function when a non-essential dependency is unavailable.
- Every load balancer or traffic router fronting a production workload SHALL use health probes that exercise the dependencies needed by the application, and unhealthy instances MUST be removed from rotation automatically.
- Every message-processing component in a production workload SHALL handle duplicate delivery safely, so that retried operations are idempotent.

Reliability testing

- Every production workload SHALL undergo fault-injection or chaos testing against its critical flows at least annually, and the results MUST be documented with follow-up actions.
- Every production workload SHALL be load tested before launch and after any change that materially alters capacity, to confirm that scaling behaves as designed.

Backup and disaster recovery

- Every production workload SHALL maintain a documented disaster recovery plan covering every component and the system as a whole, aligned to the workload RTO and RPO.

- The disaster recovery plan of a production workload MUST define the activation criteria that distinguish a disaster from a minor or major incident, and it MUST name the role authorized to declare a disaster.
- The disaster recovery plan of a production workload SHALL be exercised by a DR drill, including failover and failback, at least annually.
- Every production data store SHALL have automated backups with a retention period defined by business requirements, and at least one backup copy MUST be stored in a location isolated from the primary failure domain.
- Backup restores for every production data store SHALL be tested at least twice a year, and the measured restore time MUST be compared with the workload RTO.

Health monitoring

- Every production workload SHALL maintain a health model that maps component health signals to the health of each critical flow.
- Every production workload SHALL collect availability and reliability telemetry from all critical-flow components and MUST retain it long enough to support post-incident analysis.

Sources

- https://learn.microsoft.com/azure/well-architected/reliability/checklist
- https://learn.microsoft.com/azure/well-architected/reliability/metrics
- https://learn.microsoft.com/azure/well-architected/reliability/disaster-recovery

Chapter 2 — Security Posture and Threat Protection

Intent

Security failures expose confidential data, damage integrity, and interrupt service. This chapter requires a measurable security baseline, classified data, strong encryption, hardened components, protected secrets, and monitoring and response capability that assume an attacker will eventually succeed. Layered controls limit the blast radius of any single failure.

Requirements

Baseline and secure development

- Every production workload SHALL be assessed against a documented security baseline that is aligned with applicable compliance obligations and with Azure platform recommendations.
- Every production workload SHALL measure its configuration against the security baseline at least quarterly, and every deviation MUST be tracked to closure or to a documented, approved exception.
- Every software development team building production workloads SHALL follow a documented secure development lifecycle that includes threat modeling during design.
- Every build pipeline for a production workload MUST include automated static code analysis and dependency vulnerability scanning, and it SHALL block release when a critical-severity finding is open.
- Every source repository for a production workload SHALL have secret scanning enabled, and credentials MUST NOT be committed to source control.

Data classification and encryption

- Every data store and data-processing system in a production workload SHALL be assigned a sensitivity classification and an information-type label.
- The classification of a data set MUST determine the minimum encryption, retention, and access controls applied to it.
- All production data at rest SHALL be encrypted with platform-native encryption using industry-standard algorithms.
- All production data in transit SHALL be encrypted with TLS 1.2 or later, and plaintext protocols MUST NOT be used between production components.
- Data classified at the highest sensitivity level SHOULD be protected with customer-managed keys held in Azure Key Vault or Azure Managed HSM.
- Every customer-managed encryption key SHALL have a documented rotation schedule and MUST be recoverable through soft delete and purge protection.
- Production workloads SHALL NOT disable platform-provided encryption in order to implement custom cryptography, and custom cryptographic algorithms MUST NOT be created.

- Every case where encryption of a production data store is not applied SHALL have a documented technical or business justification approved by the security owner.

Hardening and patching

- Every compute, data, and platform resource in a production workload SHALL be deployed from a hardened configuration that disables unused features, ports, accounts, and services.
- Every virtual machine in a production workload SHALL receive operating-system security updates through an automated patching process, and critical updates MUST be applied within 14 days of release.
- Every container image deployed to production SHALL be built from an approved base image and scanned for vulnerabilities, and images with unremediated critical vulnerabilities MUST NOT be deployed.
- Every production Azure subscription SHOULD have Microsoft Defender for Cloud enabled with its recommendations triaged at least monthly.

Secrets management

- Every application secret in a production workload SHALL be stored in a dedicated secret store such as Azure Key Vault, and secrets MUST NOT be stored in source code, container images, or plain-text configuration files.
- Every production workload SHOULD use managed identities in place of stored credentials wherever the target service supports them.
- Every production secret SHALL have a defined rotation interval, and the secret management process MUST support emergency rotation outside the schedule.
- Every consumer of a preshared key SHALL receive its own distinct key, and the same key MUST NOT be used in both production and non-production environments.
- Every access to production secrets and every change to them SHALL be written to an audit log that is retained and monitored.

Threat detection and monitoring

- Every production workload SHALL forward security-relevant logs from compute, identity, network, and data tiers to a central security log store.
- Every production workload SHALL have threat detection enabled that raises alerts into the organization's security operations triage process.
- Every security alert rule SHALL have a named owner and MUST be reviewed at least annually for relevance and false-positive rate.
- Security log data for production workloads MUST be retained for at least 12 months, or longer where a compliance obligation requires it.

Security testing and incident response

- Every production workload SHALL undergo penetration testing at least annually and after significant architectural change.

- Every production workload SHALL test its threat detection rules at least annually by simulating attacker behavior.
- Every production workload SHALL have a documented security incident response plan that names the team or individual responsible for each procedure.
- The security incident response plan of a production workload MUST be rehearsed at least annually, and lessons learned SHALL be recorded and fed back into the baseline.

Sources

- https://learn.microsoft.com/azure/well-architected/security/checklist
- https://learn.microsoft.com/azure/well-architected/security/encryption
- https://learn.microsoft.com/azure/well-architected/security/application-secrets

Chapter 3 — Identity and Access Management

Intent

Identity is the primary security perimeter for cloud workloads, covering people, services, and anonymous callers alike. This chapter requires one trusted identity provider, strong and conditional authentication, least-privilege authorization, and auditable access. Together these controls ensure that every action is attributable and limited to what is necessary.

Requirements

Identity provider and authentication

- Every production workload SHALL authenticate all human and system identities through Microsoft Entra ID or another centrally managed identity provider approved by the organization.
- Production workloads MUST NOT implement custom user stores or custom authentication mechanisms in place of the approved identity provider.
- Every production workload SHALL use modern authentication and authorization protocols such as OpenID Connect and OAuth 2.0, and legacy protocols such as basic authentication MUST NOT be enabled.
- Every production workload SHALL document both outside-in access (users and operators reaching the workload) and inside-out access (the workload reaching other resources), together with the identity used for each flow.
- Every production workload SHALL define which resources are accessed anonymously, and anonymous access MUST be limited to resources that are intended to be public.

Multifactor and conditional access

- Every interactive human account with access to production resources SHALL be protected by multifactor authentication.
- Every administrative role assignment on production resources MUST require phishing-resistant or conditional-access-enforced multifactor authentication.
- Production access policies SHOULD apply Conditional Access that evaluates user risk, sign-in risk, device compliance, and location before granting access.
- Legacy authentication protocols SHALL be blocked by Conditional Access policy for all accounts with access to production resources.
- Break-glass emergency accounts SHALL be limited to at most two per tenant, MUST be excluded from normal Conditional Access dependencies only by documented design, and SHALL have every sign-in alerted.

Authorization and least privilege

- Every role assignment on production Azure resources SHALL follow least privilege and be scoped to the narrowest resource, resource group, or subscription that meets the need.

- Role assignments for production resources SHOULD be made to groups rather than individual users, so that access follows team membership.
- Every production workload SHALL separate control-plane roles from data-plane roles and MUST NOT grant control-plane permissions to identities that need only data access.
- The Owner and User Access Administrator roles on production subscriptions SHALL be assigned to no more than the number of named individuals documented by the platform team, and MUST NOT be assigned to workload service identities.
- Custom Azure roles SHALL be created only when no built-in role satisfies the requirement, and each custom role MUST have a documented owner and justification.
- Every subscription and management group SHALL have Azure RBAC assignments reviewed against actual need at least semiannually.

Privileged access

- Standing privileged access to production resources SHOULD be replaced with just-in-time elevation using Microsoft Entra Privileged Identity Management.
- Every privileged elevation on production resources SHALL be time-bound with a maximum activation duration of 8 hours and MUST require a recorded business justification.
- Administrators SHALL use separate, dedicated accounts for privileged operations and MUST NOT use those accounts for email or general web browsing.

Workload and resource identities

- Every workload component that authenticates to an Azure service SHALL use a managed identity or workload identity federation, and static client secrets MUST NOT be used where those options are available.
- Every workload component SHALL have its own distinct identity, and an identity MUST NOT be shared across unrelated workloads or across production and non-production environments.
- Where a service principal secret or certificate cannot be avoided, the credential SHALL expire within 12 months and be rotated before expiry.
- Shared access signature tokens SHALL be time-limited, scoped to the narrowest permission, and issued by user delegation where the service supports it.
- Storage account shared-key access and similar key-based authorization SHOULD be disabled on production resources that support identity-based authorization.

Lifecycle and audit

- Every human identity with access to production resources SHALL be removed or disabled within 24 hours of the person leaving the organization or the role requiring the access.
- Every guest or external identity in the tenant SHALL be sponsored by a named internal owner and reviewed at least quarterly.
- Every production workload SHALL log all sign-ins, authorization decisions, and role-assignment changes to a central store retained for at least 12 months.

- Every access review of production role assignments MUST record the reviewer, the date, and the decision for each assignment reviewed.

Sources

- https://learn.microsoft.com/azure/well-architected/security/identity-access
- https://learn.microsoft.com/azure/well-architected/security/checklist
- https://learn.microsoft.com/azure/well-architected/security/application-secrets

Chapter 4 — Network Architecture and Edge Protection

Intent

Network controls add defense in depth to identity controls by hiding resources, filtering traffic, and limiting lateral movement. This chapter requires deliberate segmentation, a well-defined hub-and-spoke or equivalent topology, strict filtering of inbound and outbound flows, and private connectivity to platform services. The goal is a small, well-monitored public surface and contained internal movement.

Requirements

Traffic classification and topology

- Every production workload SHALL classify each network flow by direction (ingress or egress), scope (north-south or east-west), and visibility (public or private) in its design documentation.
- Every Azure landing zone network design SHALL use a documented topology, such as hub-and-spoke or Azure Virtual WAN, with shared connectivity services placed in a dedicated connectivity subscription.
- Workloads requiring internet exposure SHOULD be placed in subscriptions separate from workloads that are reachable only through private or hybrid connectivity.
- Every virtual network address space SHALL be allocated from an enterprise IP address plan and MUST NOT overlap with any other virtual network or on-premises range that it may connect to.

Segmentation

- Every production workload SHALL be deployed in virtual networks dedicated to that workload or its landing zone, and production and non-production resources MUST NOT share a virtual network.
- Every virtual network in a production workload SHALL be divided into subnets by tier or security requirement, and each subnet MUST have a network security group attached.
- Every network security group SHALL end with a rule that denies all traffic not explicitly allowed, and allow rules MUST specify the narrowest practical source, destination, and port.
- Network security group rules MUST NOT allow any-to-any traffic, and inbound rules from the internet to management ports such as RDP (3389) and SSH (22) SHALL NOT exist.
- Virtual network peerings SHALL be created only between networks that have a documented communication need, and forwarded traffic and gateway transit MUST be enabled only where the design requires it.
- Application security groups or service tags SHOULD be used in network security group rules instead of hard-coded IP addresses where the platform supports them.

Edge and ingress protection

- Every public HTTP or HTTPS workload SHALL be exposed through a reverse-proxy or gateway such as Azure Application Gateway or Azure Front Door, and backend compute MUST NOT have public IP addresses.
- Every internet-facing web application SHALL be protected by a web application firewall in prevention mode with the managed rule set enabled.
- Every internet-facing workload SHALL be protected by Azure DDoS Protection at the network or IP-protection tier.
- Every public endpoint SHALL accept only TLS 1.2 or later and MUST redirect or reject plaintext HTTP requests.
- Every public IP address in production SHALL have a documented owner and business purpose, and unused public IP addresses MUST be removed.
- Remote administration of production virtual machines SHALL be performed through Azure Bastion or an equivalent just-in-time access method, and direct inbound management access MUST NOT be used.

Egress control

- All internet-bound outbound traffic from production virtual networks SHALL be routed through a central egress control such as Azure Firewall or an approved network virtual appliance using user-defined routes.
- Egress rules in production SHALL permit only documented destinations, and a default allow-all outbound rule MUST NOT be used.
- Azure Firewall policies SHOULD enable threat-intelligence filtering in alert-and-deny mode and, for workloads handling sensitive data, SHOULD enable TLS inspection and intrusion detection.
- Every firewall rule change in production SHALL follow a documented change approval process, and firewall rules MUST be reviewed at least semiannually.

Private connectivity and DNS

- Every production Azure PaaS data service holding sensitive data SHALL be reached through Private Link private endpoints, and public network access on that service MUST be disabled.
- Every production Azure virtual network that uses private endpoints SHALL resolve them through Azure Private DNS zones linked to that network or to a central DNS resolver.
- Hybrid connectivity for production workloads SHALL use ExpressRoute or IPsec site-to-site VPN, and ExpressRoute circuits serving critical flows SHOULD be paired with a redundant path in a distinct location.
- Traffic between production components in different regions or networks SHALL traverse the Azure backbone or a private connection and MUST NOT traverse the public internet unencrypted.

Monitoring

- Every network security group and Azure Firewall instance in production SHALL send flow or diagnostic logs to a central Log Analytics workspace, and these logs MUST be retained for at least 90 days.
- Every production virtual network SHOULD have Azure Network Watcher enabled and traffic analytics reviewed at least monthly.

Sources

- https://learn.microsoft.com/azure/well-architected/security/networking
- https://learn.microsoft.com/azure/cloud-adoption-framework/ready/landing-zone/design-area/network-topology-and-connectivity
- https://learn.microsoft.com/azure/well-architected/security/checklist

Chapter 5 — Data, Storage and Privacy

Intent

Data is the asset most likely to cause lasting harm when it is exposed, altered, or lost. This chapter sets obligations for classifying data, protecting it at rest and in transit, managing encryption keys, and keeping it recoverable. Applying these obligations consistently lets each data store receive protection proportionate to the sensitivity of its contents.

Requirements

Data classification and ownership

- Every workload data store and every system that processes workload data SHALL carry a documented sensitivity classification drawn from the organization's classification taxonomy. Classification drives the protection level that is applied to the data.
- The design document MUST record the location of every store that holds classified data, including copies in backups, caches, analytics stores, and nonproduction environments. Data that is not located cannot be protected.
- Each data store MUST have a named owner who is accountable for its classification, access approvals, and retention. Ownership prevents orphaned data from accumulating without controls.
- Automated data discovery tooling SHOULD be used to suggest classifications, and a human review of the suggested classifications MUST occur before they are accepted. Tooling alone does not replace team accountability.
- Production data classified as confidential or restricted SHALL NOT be copied into development or test environments unless it has first been masked or anonymized.

Encryption at rest

- All data at rest in production storage, including databases, storage accounts, managed disks, and backups, MUST be encrypted. Platform-native encryption SHOULD be used instead of custom encryption.
- Teams SHALL NOT disable platform-provided encryption in order to substitute a custom implementation unless a documented evaluation shows the platform capability cannot meet a stated requirement.
- Any custom cryptography that is permitted MUST use industry-standard algorithms and the cryptography APIs of the operating system or runtime rather than bespoke or nonplatform libraries.
- Data classified as confidential or restricted SHOULD be protected with customer-managed keys when regulation or policy requires the organization to control key rotation, revocation, or the ability to render data undecryptable.
- Any data store that is not encrypted at rest MUST have a documented, approved exception that states the technical or business reason.

- Azure SQL Database and Azure SQL Managed Instance SHALL have transparent data encryption enabled. Sensitive columns SHOULD additionally be protected with Always Encrypted when the database administrator is not trusted to read them.

Encryption in transit

- All network traffic that carries workload data between clients and services, and between services, MUST be encrypted by using TLS 1.2 or later. Earlier protocol versions SHALL NOT be enabled.
- Azure Storage accounts MUST require secure transfer so that unencrypted HTTP requests are rejected. Database endpoints such as Azure SQL and Azure Cosmos DB MUST be accessed only over encrypted connections.
- Certificates used for data-in-transit protection MUST be issued by a trusted authority, stored in a managed certificate store such as Azure Key Vault, and renewed automatically or alerted on before expiry.

Key and secret management

- Encryption keys, connection strings, and other secrets MUST be stored in a dedicated managed service such as Azure Key Vault or Managed HSM, and SHALL NOT be stored in source code, configuration files, or container images.
- Encryption keys MUST be rotated according to a documented schedule, and rotation SHOULD be automated. A compromised key MUST be revoked and replaced within the period defined by the incident response plan.
- Key vaults that protect production keys MUST have soft delete and purge protection enabled. This prevents a deleted key from making encrypted data permanently unrecoverable.
- Workloads SHOULD authenticate to data stores by using managed identities with Microsoft Entra ID rather than shared keys or passwords. Where shared keys remain, key-based access MUST be justified in writing.

Access and exposure

- Production data stores SHALL NOT be reachable from the public internet unless a documented business need exists. Private endpoints or service endpoints with firewall rules SHOULD be used for Azure storage and database services.
- Access to data stores MUST follow least privilege, using role-based access control scoped to the narrowest practical resource and granted only to the identities that need it.
- Storage containers MUST NOT allow anonymous public read access unless the content is explicitly classified as public.

Backup, retention and recovery

- Every production data store MUST have a backup strategy that states the recovery point objective and recovery time objective it satisfies. The strategy SHALL be reviewed at least annually.

- Backups of production data MUST be encrypted and stored in a location isolated from the primary data, such as a separate region or a separate account with restricted access.
- Restoration from backup MUST be tested at least annually for each production data store, and the outcome of each test SHALL be recorded.
- Each data type MUST have a defined retention period, and data SHALL be deleted or archived when that period expires. Personal data SHALL NOT be retained longer than the stated purpose requires.
- Azure Blob Storage SHOULD have soft delete or versioning enabled for production containers, and immutable storage MUST be used for data subject to legal or regulatory write-once requirements.

Data residency and privacy

- Data subject to residency requirements MUST remain in the approved regions, and the design document SHALL state the region for every store and replica that holds such data.
- Personal data MUST be collected only for a documented purpose, and the design SHALL describe how access, correction, and deletion requests from data subjects are fulfilled.

Sources

- https://learn.microsoft.com/azure/well-architected/security/encryption
- https://learn.microsoft.com/azure/well-architected/security/data-classification
- https://learn.microsoft.com/azure/cloud-adoption-framework/ready/azure-best-practices/resource-tagging

Chapter 6 — Monitoring, Logging, Tracing and SRE

Intent

Teams can only operate what they can observe. This chapter sets obligations for telemetry design, log handling, alerting, service-level objectives, distributed tracing, and incident response so that degradation is detected early and diagnosed quickly. It treats the monitoring stack as a first-class part of the workload rather than an afterthought.

Requirements

Health model and telemetry design

- Every workload SHALL define a health model that lists its health indicators, key performance indicators, and the metrics that feed them. Telemetry collection MUST be derived from that model.
- The health model MUST map user flows and system flows to the resources that support them, so that degradation in a dependency is visible at the level where business impact occurs.
- Workload components MUST emit structured telemetry that uses a consistent schema and includes the source component, a timestamp, and a correlation identifier.
- Applications SHOULD use a configurable logging framework that is decoupled from any specific monitoring backend, so the backend can change without code changes.

Logging practices

- Logs MUST NOT contain secrets, credentials, or unmasked personal or sensitive data. Applications SHALL scrub such values before writing log entries.
- Telemetry MUST be classified by operational concern, such as audit, security, debugging, and performance, so that access controls and retention can be applied per category.
- Workload business data SHALL NOT be stored in telemetry stores. Telemetry and workload data MUST be kept separate.
- Production logging verbosity SHOULD be lower than development verbosity and MUST be adjustable at runtime or by configuration without redeployment.
- Telemetry emission MUST be non-blocking and MUST NOT cause the application to fail when the monitoring backend is unavailable. The only exception is critical audit logging that is required by regulation.

Azure Monitor and Application Insights

- Every production Azure resource MUST send platform logs and metrics to a Log Analytics workspace through diagnostic settings. Diagnostic settings SHOULD be deployed by Azure Policy so that new resources are covered automatically.
- Every production application MUST be instrumented with Application Insights, or an OpenTelemetry-based equivalent that exports to Azure Monitor, so that requests, dependencies, exceptions, and performance counters are captured.

- Workspace-based Application Insights resources SHOULD be used so that application and infrastructure telemetry can be queried together.
- Access to log workspaces MUST be controlled by role-based access control, and workspaces holding security or audit data SHALL be restricted to authorized personnel.

Retention

- The retention period for each log category MUST be defined in the design document and SHALL satisfy the longest applicable regulatory or organizational requirement.
- Audit and security logs SHOULD be kept for at least one year, using archive or long-term storage tiers for the period beyond interactive retention to control cost.
- Log data MUST be protected from alteration and deletion for the duration of its retention period.

Distributed tracing and sampling

- Every request that crosses a service boundary MUST propagate a correlation identifier, using the W3C Trace Context standard, so that a transaction can be traced end to end.
- Services in a distributed system SHALL emit trace spans for all inbound requests and outbound dependency calls.
- Sampling, when used for traces or telemetry in production, MUST preserve all failed requests and all exceptions. The sampling rate SHALL be documented, and sampling decisions MUST be consistent across the services in a single transaction.

Alerting

- Every alert MUST have a named owning team, a severity, and a linked runbook or response procedure. Alerts without an owner SHALL NOT be enabled in production.
- Alerts MUST be based on symptoms that affect users or on breach of a service-level objective, and SHOULD NOT fire on causes that require no human action.
- Production alert rules and action groups MUST be defined as code and stored in version control. Alerting MUST include at least one notification path that is independent of the monitored workload.
- Alert rules SHALL be reviewed at least quarterly, and rules that are noisy or never actionable MUST be tuned or removed.

SLIs, SLOs and error budgets

- Every production service MUST define at least one service-level indicator for availability and one for latency, each with a documented measurement method.
- Every production service MUST have a service-level objective for each indicator, agreed with the service's business owner, with a stated measurement window.
- Teams SHOULD track an error budget derived from each objective and MUST define the actions taken, such as pausing feature releases, when the budget is exhausted.

Incident response

- Every production workload MUST have a documented incident response process that defines severity levels, on-call responsibilities, escalation paths, and communication procedures.
- On-call engineers MUST have access to dashboards, logs, and traces sufficient to diagnose an incident without requesting additional access during the incident.
- A blameless post-incident review MUST be completed within 10 business days after every incident at the highest two severity levels, and SHALL record corrective actions with owners and due dates.

Sources

- https://learn.microsoft.com/azure/well-architected/operational-excellence/observability
- https://learn.microsoft.com/azure/well-architected/operational-excellence/safe-deployments

Chapter 7 — Operational Excellence, CI/CD and Infrastructure as Code

Intent

Reliable operations depend on repeatable, automated, and reviewed change. This chapter sets obligations for infrastructure as code, pipelines, environment separation, deployment safety, secret handling, documentation, and change management. Following them reduces human error, configuration drift, and the blast radius of faulty releases.

Requirements

Infrastructure as code

- All infrastructure and configuration changes to production environments MUST be made through infrastructure as code and deployed through a CI/CD pipeline. Manual changes in the Azure portal SHALL NOT be made to production except during a documented emergency.
- Infrastructure as code SHOULD use a declarative approach, such as Bicep or Terraform, in preference to imperative scripts.
- Infrastructure as code definitions MUST be stored in version control and reviewed through pull requests before they are merged to the deployment branch.
- Infrastructure as code MUST be organized into modules with defined inputs and outputs, and module versions SHALL be pinned.
- Deployment code SHALL NOT reference the latest version of a tool, API, provider, or module. Every such reference MUST name an explicitly tested version.
- The organization SHOULD maintain a style guide that standardizes the infrastructure as code tools, naming, and structure used by all teams.
- Configuration drift between deployed resources and their code definitions MUST be detected on a recurring schedule, and detected drift SHALL be corrected through code rather than manual edits.

Environments

- Every workload MUST have at least one nonproduction environment that matches production in topology and configuration, so that releases are validated before they reach production.
- All environments of a workload SHALL be created from the same infrastructure as code definitions, differing only by parameter values.
- Production and nonproduction environments MUST be isolated by separate subscriptions or resource groups, with separate identities and access assignments.

CI/CD pipelines

- Every production deployment MUST be performed by an automated pipeline, and the pipeline definition SHALL be stored in version control.

- Pipelines MUST run automated build, unit test, static analysis, and dependency vulnerability scanning stages before an artifact can be promoted.
- A single build artifact MUST be produced once and promoted unchanged through every environment, rather than being rebuilt per environment.
- Pipeline identities MUST use least-privilege permissions scoped to the target environment, and SHOULD authenticate to Azure by using workload identity federation instead of stored credentials.
- Production deployments MUST require approval from at least one person other than the author of the change.

Secrets in pipelines

- Secrets SHALL NOT be committed to source control, embedded in pipeline definitions, or printed in pipeline logs. Pipelines MUST retrieve secrets at run time from a secret store such as Azure Key Vault or the pipeline platform's protected secret storage.
- Source repositories MUST have secret scanning enabled, and any credential detected in a repository MUST be rotated immediately.
- Pipeline service connections and tokens MUST have a defined owner and SHALL be reviewed at least annually.

Branching and code review

- The default branch of every workload repository MUST be protected so that direct pushes are rejected and changes require a reviewed pull request with passing status checks.
- Teams SHOULD use short-lived branches and integrate to the default branch at least daily, so that changes remain small and incremental.

Deployment safety

- Every deployment, including code, infrastructure, feature flag, and configuration changes, MUST follow the same standardized, automated process. No category of production change SHALL be treated as exempt from risk management.
- Production releases MUST use progressive exposure, such as canary, blue-green, or ring-based rollout, so that a faulty release affects only a subset of users before it is promoted.
- A health check MUST pass before a deployment advances to each subsequent stage of progressive exposure, and the deployment MUST halt automatically when health checks fail.
- Every production deployment MUST have a tested rollback or roll-forward procedure, and the procedure SHALL be exercised at least annually.
- Emergency or hotfix deployments MUST have a documented expedited process that still uses the pipeline and retains an audit record.

Documentation and decision records

- Significant architectural decisions MUST be recorded as architecture decision records that state the context, the decision, the alternatives considered, and the consequences. Records SHALL be stored in the repository alongside the code.
- Each production workload MUST maintain runbooks for routine operations and recovery procedures, and the runbooks SHALL be reviewed at least annually.

Change management

- Every change to a production workload MUST be traceable to a work item and a commit, so that the reason for any production change can be audited.
- Changes classified as high risk MUST have a documented risk assessment, a defined maintenance window, and a communication plan before deployment.

Sources

- https://learn.microsoft.com/azure/well-architected/operational-excellence/infrastructure-as-code-design
- https://learn.microsoft.com/azure/well-architected/operational-excellence/safe-deployments

Chapter 8 — Cost Optimization and FinOps

Intent

Cloud spend grows with every design decision, so cost must be modeled, tracked, and governed continuously rather than corrected after the fact. This chapter sets obligations for cost models, budgets, tagging, allocation, rightsizing, commitment discounts, and scaling. A cost-optimized workload delivers its required value at a justified price, which is not always the lowest price.

Requirements

Cost model and accountability

- Every workload MUST have a documented cost model that estimates initial cost, run rate, and ongoing cost, and that includes projected growth. The model SHALL be reviewed at least annually and after any significant design change.
- Every workload MUST have a named owner who is accountable for its spending, and the design document SHALL identify that owner.
- The design document SHOULD state the cost implications of significant architectural and technology choices, including build-versus-buy and licensing models.
- Each workload budget MUST include a buffer for unplanned spending in addition to the cost model estimate.

Budgets and alerts

- Every production subscription or resource group MUST have an Azure Cost Management budget with notification thresholds. Alerts MUST be sent to the accountable owner.
- Budget alerts MUST include at least one threshold below 100 percent of the budget and one threshold on forecasted spend.
- Cost anomaly detection or alerting SHOULD be enabled so that deviations from the spending trend are surfaced within one day.
- Cost data MUST be collected daily and reviewed by the workload owner and stakeholders at least monthly, comparing actual spend with the budget and cost model.
- Cost reports SHOULD show both incurred costs and amortized costs of prepaid commitments, along with trends and forecasts.

Tagging and cost allocation

- Every billable Azure resource MUST carry the organization's mandatory tags, which SHALL include at least cost center or owner, workload name, and environment.
- Mandatory tags MUST be enforced by Azure Policy using deny or append and inherit effects, so that untagged resources are prevented or remediated.
- Tag names and permitted values MUST be defined in a central tagging standard, and teams SHALL NOT create variants of mandatory tag names.

- Tag inheritance SHOULD be enabled in Cost Management, so that costs of resources are attributed to the tags set at the subscription or resource group level.
- Shared platform costs MUST be allocated to consuming workloads by a documented allocation rule rather than being left unassigned.

Showback and chargeback

- The organization MUST produce showback reports at least monthly that show each business unit or workload owner the cost it incurred.
- Where chargeback is practiced, the allocation method MUST be documented, approved by finance, and applied consistently across all consuming teams.

Guardrails

- Spending guardrails MUST be implemented through platform automation, such as Azure Policy restrictions on permitted SKUs and regions, rather than by manual review alone.
- Resource creation rights in production MUST be restricted to approved roles or pipelines, so that unmanaged resources cannot generate unbudgeted cost.
- Nonproduction environments SHOULD use lower-cost SKUs than production and SHALL be shut down or scaled to zero outside working hours when they are not needed.

Rightsizing and lifecycle

- Resource sizes MUST be reviewed against measured utilization at least quarterly, and resources whose sustained utilization is below the documented threshold SHALL be downsized or removed.
- Orphaned resources, such as unattached disks, unused public IP addresses, and idle gateways, MUST be identified and deleted on a recurring schedule.
- Data in Azure Storage SHOULD be moved to cooler access tiers through lifecycle management policies when its access frequency declines.
- Log and telemetry volume MUST be reviewed periodically, and low-value data SHOULD be filtered, sampled, or moved to a lower-cost tier.

Reservations, savings plans and licensing

- Workloads with steady, predictable usage SHOULD be covered by Azure Reservations or Azure savings plans, with coverage reviewed at least annually.
- Commitments MUST NOT be purchased until at least one representative period of utilization data supports the commitment size.
- Existing eligible licenses SHOULD be applied through Azure Hybrid Benefit, and the organization MUST track license entitlements to avoid paying twice.
- Development and test workloads SHOULD use Azure Dev/Test pricing where the organization's agreement permits.

Autoscaling

- Every production compute service with variable demand MUST use autoscaling with defined minimum and maximum limits, so that capacity follows load without exceeding the budget.
- Autoscale rules SHALL be based on measured metrics, and the maximum instance count MUST be set to a value that fits within the approved budget.
- Autoscale and scaling-to-zero behavior MUST be tested under load before production release, so that cost savings do not compromise availability objectives.

Sources

- https://learn.microsoft.com/azure/well-architected/cost-optimization/principles
- https://learn.microsoft.com/azure/well-architected/cost-optimization/checklist
- https://learn.microsoft.com/azure/cloud-adoption-framework/ready/azure-best-practices/resource-tagging

Chapter 9 — Compute, Containers and Hosting

Intent

Compute and container hosting decisions determine how a workload scales, recovers, and is patched for its entire life. Choosing the wrong platform, running too few instances, or shipping unverified images creates availability and security debt that is expensive to reverse. This chapter sets baseline obligations for selecting hosting platforms and for operating Azure Container Apps, Azure Kubernetes Service (AKS), Azure App Service and Azure Functions in production.

Requirements

Hosting platform selection

- Every workload design document SHALL record the compute platform decision, including the alternatives considered, by reference to the Azure compute decision tree. This makes the rationale for the platform reviewable.
- A design that selects Azure Container Apps express for a production workload MUST document that the workload does not need fine-grained control over compute, networking, or cold-start behavior. Workloads that need that control SHOULD use Container Apps with a workload profiles environment.
- A design that selects AKS for a new production workload SHOULD use AKS Automatic and MUST document the specific requirement, such as Windows node pools or bring-your-own CNI, that justifies AKS Standard when AKS Standard is chosen.
- Every hosting design MUST state which team owns each part of the platform, with the platform team owning cluster or plan configuration and the application team owning manifests, application identity, and workload monitoring.
- Unrelated workloads SHOULD run in separate AKS clusters. A shared AKS cluster MUST use namespaces, dedicated node pools, and network policies to enforce isolation between the workloads it hosts.

Redundancy and availability

- Every production Container Apps app exposed through HTTP or TCP ingress SHALL run at least three replicas. Apps that are sensitive to cold starts MUST configure a minimum number of always-ready replicas.
- Every production compute deployment in a region that offers availability zones MUST enable zone redundancy for Container Apps environments, AKS node pools, and App Service plans. This keeps a single zone failure from taking the workload down.
- A production App Service app SHALL run on at least two instances. A single-instance deployment is a single point of failure.
- Compute for a critical workload MUST be deployed to at least two Azure regions, with Azure Front Door or Azure Traffic Manager routing traffic between them.

- Every production AKS cluster SHALL place application pods on user node pools, with at least three nodes in the system node pool and at least two nodes in each user node pool. System pods are thereby isolated from application resource demands.
- Every production AKS workload with more than one replica MUST define a pod disruption budget. The budget keeps node upgrades and other voluntary disruptions from removing too many pods at once.
- Stateful container workloads MUST store persistent data on volumes outside the container, and in a zone-redundant region the volumes SHOULD use zone-redundant storage.

Health probes and self-healing

- Every container in a production deployment MUST define startup, readiness, and liveness probes. Without them the platform cannot restart failed containers or withhold traffic from containers that are not ready.
- Liveness probes MUST test only the health of the container itself and MUST NOT check downstream dependencies. A dependency outage would otherwise trigger needless restarts across every replica.
- Readiness probe and startup probe settings SHALL allow enough time for the slowest expected initialization. Probe thresholds that restart a slow-starting container prematurely MUST NOT be used.
- Applications that call other services SHOULD use platform-level resiliency policies, such as Container Apps retries, timeouts, and circuit breakers, in addition to application-level handling.

Scaling and resource limits

- Every production container app and Functions deployment SHALL use automatic horizontal scaling with explicit scale rules, such as HTTP concurrency, queue depth, or CPU and memory thresholds, rather than manual instance counts.
- Autoscaling rules MUST be load tested before production release, and scale events SHALL be logged so scaling behavior can be reviewed.
- Every container in production MUST declare CPU and memory requests and limits. Limits prevent a single container from starving neighbors and allow usage to be tuned from monitored data.
- Session or application state SHALL NOT be held only in the memory of a single compute instance. State that must survive restarts or scale-out MUST live in an external store that scales independently.
- Azure Functions that process queue or event triggers MUST be idempotent and MUST use built-in retry policies for transient errors. Long-running, multi-step workflows SHOULD use Durable Functions.

Image provenance and patching

- Production container images MUST be pulled only from a private registry that the organization controls, such as Azure Container Registry. Images from public registries SHALL first be imported and scanned.
- Every container image deployed to production MUST be scanned for vulnerabilities before release and on a recurring schedule afterward. Images with unresolved critical vulnerabilities MUST NOT be deployed.
- Deployments MUST reference container images by immutable digest or a unique, non-reused version tag. The mutable `latest` tag SHALL NOT be used in production.
- Container images SHOULD be built from minimal, regularly refreshed base images and SHOULD be signed so that deployment can verify their origin.
- Every AKS cluster and App Service plan MUST have a defined patching and upgrade process, including node image updates, that is exercised at least monthly and that honors pod disruption budgets.

Observability

- Every production compute resource SHALL emit logs and metrics to Azure Monitor and SHOULD emit distributed traces through OpenTelemetry. Alerts MUST exist for availability, restarts, and scaling saturation.

Sources

- https://learn.microsoft.com/azure/well-architected/service-guides/azure-container-apps
- https://learn.microsoft.com/azure/well-architected/service-guides/azure-kubernetes-service
- https://learn.microsoft.com/azure/well-architected/service-guides/app-service-web-apps
- https://learn.microsoft.com/azure/well-architected/service-guides/azure-functions-security

Chapter 10 — Services, APIs and Messaging

Intent

Services communicate over networks that fail, throttle, and deliver messages more than once. Well-designed APIs, bounded retries, and durable messaging keep one component's failure from cascading and let producers and consumers evolve independently. This chapter sets obligations for API design and versioning, service-to-service calls, and asynchronous messaging on Azure.

Requirements

API design

- Every web API SHALL model its URIs on resources named with nouns, and collection URIs MUST use plural nouns. Verbs such as `create-order` SHALL NOT appear in resource URIs.
- Every web API MUST use HTTP methods according to their standard semantics and MUST return accurate HTTP status codes, such as 201 with a Location header for creation and 404 for missing resources.
- GET, PUT, DELETE, HEAD, and PATCH operations of every web API MUST be idempotent. A repeated request against the same resource must leave the resource in the same state.
- A POST operation that creates a resource MUST NOT cause side effects on resources unrelated to the new resource.
- Every web API operation that takes long to complete SHOULD return HTTP 202 with a status endpoint URI in the Location header, instead of holding the client connection open.
- Every web API collection endpoint MUST support pagination with documented default and maximum page sizes. The maximum limit prevents a single request from retrieving unbounded data.
- Web APIs SHOULD support content negotiation through the Accept header and MUST return 415 for unsupported request media types and 406 when no acceptable response format exists.
- Every public API MUST publish a machine-readable contract, such as an OpenAPI document, that is kept in source control with the implementation.

API versioning and gateway

- Every API consumed outside its owning team MUST carry an explicit version in the URI, a header, or a query parameter. Unversioned public endpoints SHALL NOT be released.
- A breaking change to an API contract, such as removing a field, changing a type, or changing required parameters, SHALL be released only as a new major version. The existing major version MUST remain available during a published deprecation period.
- Every API version retirement MUST be announced to consumers in advance, with a stated end date, before the version is removed.

- Every API exposed to external or cross-team consumers SHALL be published through an API gateway, such as Azure API Management, rather than directly from the backend service. The gateway applies authentication, rate limiting, and policy consistently.
- API gateway policies for every production API MUST enforce request rate limits or quotas and MUST validate inbound requests against the published contract.

Service-to-service communication

- Every outbound call to a remote service or dependency MUST have an explicit timeout. Calls without timeouts SHALL NOT be shipped, because blocked threads and connections can exhaust resources.
- Every retry policy MUST apply only to transient faults, use exponential backoff with jitter, and enforce a maximum attempt count. Retries for non-transient errors SHALL NOT be attempted.
- Retry logic SHALL be implemented at only one layer of a call chain. Lower layers MUST fail fast so that nested retries do not multiply delays and load.
- Every dependency call that can fail persistently MUST be wrapped by a circuit breaker with documented failure threshold, open duration, and half-open probe behavior. Retry logic MUST stop when the circuit breaker reports the circuit is open.
- Retried operations that are not naturally idempotent MUST carry an idempotency key or request identifier that the receiving service uses to discard duplicates.
- Early retry failures SHOULD be logged as informational entries and the final exhausted retry MUST be logged as an error, so that operators are not flooded with alerts for recovered faults.
- Each service MUST own its data store, and other services SHALL access that data only through the owning service's API or published events. Shared databases between independently deployed services SHALL NOT be used.
- Service boundaries SHOULD align with business capabilities, and each service MUST be independently deployable and versionable.
- Application code SHOULD use the official Azure SDK client libraries and their built-in retry and authentication support instead of hand-built REST calls to Azure services.

Messaging

- Every messaging consumer MUST be idempotent, because Azure Service Bus and Azure Event Grid deliver at least once. Consumers SHOULD track processed message identifiers to detect duplicates.
- Every Service Bus queue and subscription SHALL have a dead-letter queue policy and a documented process for reviewing and reprocessing dead-lettered messages. Poison messages MUST NOT block healthy message processing.
- Production Service Bus namespaces that need zone or regional redundancy SHALL use the Premium tier, with zone redundancy and Geo-Replication enabled. The design MUST document how and when a secondary region is promoted.

- Every message and event schema SHALL be versioned, and changes to published schemas MUST remain backward compatible for existing consumers or be released as a new schema version.
- Service Bus consumers MUST handle `QuotaExceeded` and other throttling errors, and SHOULD use a circuit breaker to stop sending while the condition persists.
- The messaging technology for each integration, such as Service Bus, Event Grid, or Event Hubs, MUST be chosen from the documented messaging decision criteria and the choice SHALL be recorded in the design.

Sources

- https://learn.microsoft.com/azure/architecture/best-practices/api-design
- https://learn.microsoft.com/azure/architecture/best-practices/api-implementation
- https://learn.microsoft.com/azure/architecture/patterns/retry
- https://learn.microsoft.com/azure/architecture/patterns/circuit-breaker
- https://learn.microsoft.com/azure/well-architected/service-guides/azure-service-bus

Chapter 11 — Governance, Compliance and Landing Zones

Intent

Governance determines whether an Azure estate stays secure, compliant, and manageable as the number of subscriptions and teams grows. Guardrails defined once in a management group hierarchy and enforced with Azure Policy scale far better than manual reviews. This chapter sets obligations for landing zone structure, policy, access governance, naming, tagging, and compliance evidence.

Requirements

Management groups and subscriptions

- The Azure management group hierarchy SHALL be kept flat, with no more than four levels below the tenant root management group. Deep nesting raises management overhead and obscures inherited policy.
- Management groups MUST be used to assign policy and initiatives to subscriptions with common security and compliance needs, and SHALL NOT be used to mirror the organizational chart.
- The hierarchy SHALL include a platform management group, a landing zones management group with child groups by workload type such as corp and online, and a sandbox management group for experimentation.
- Separate management groups SHALL NOT be created for production, test, and development environments. Environments SHOULD be separated by subscription within the same management group.
- Management groups SHALL NOT be created solely to model Azure regions. A location-based hierarchy MAY be used only where data residency or sovereignty regulation requires it.
- A default management group other than the tenant root MUST be configured for new subscriptions, so no subscription is created directly under the root.
- Azure RBAC authorization for management group operations MUST be enabled in the hierarchy settings so that only privileged principals can create or modify management groups.
- Every new workload landing zone MUST be created through an automated subscription vending process that applies baseline policy, role assignments, tags, and networking.
- Each workload SHALL be assigned its own subscription or subscriptions. Workloads with different security, compliance, or scale boundaries MUST NOT share a subscription.

Azure Policy

- Compliance and security conventions, including allowed regions, allowed resource types, and required diagnostic settings, MUST be enforced with Azure Policy at management group or subscription scope instead of by manual review.

- The number of Azure Policy assignments at the tenant root management group SHALL be kept to a minimum. Root-scope assignments MUST be limited to controls that apply to every subscription.
- Policy definitions SHALL be written once and reused through initiatives at multiple inherited scopes, and all definitions and assignments MUST be managed as code in source control.
- Every new policy MUST be rolled out first in audit mode or at a limited scope and reviewed for impact before its effect changes to deny or enforce.
- Every non-compliant resource that Azure Policy reports MUST be remediated or covered by an approved exception, and remediation tasks SHOULD be used to correct existing resources automatically.

Access governance

- Workload team members SHALL NOT receive RBAC role assignments at management group scope. Their roles MUST be assigned at the subscription or resource group scope that they need.
- Standing RBAC assignments at management group scope for platform teams SHOULD be eligible, not permanent, through Microsoft Entra Privileged Identity Management with activation limited to the time needed.
- Every RBAC role assignment SHALL follow least privilege, and assignments MUST be made to groups rather than to individual users where practical.
- Access to every production subscription MUST be reviewed at least every six months, and assignments that are no longer needed MUST be removed.

Naming and tagging

- The organization SHALL define and publish one resource naming convention that specifies the ordering of components such as resource type, workload, environment, region, and instance number. Every Azure resource MUST follow it.
- Resource names MUST NOT contain sensitive, personal, or confidential information, because names and tags are visible to many services and are not treated as customer data.
- Every Azure resource that supports tags MUST carry a defined mandatory tag set that covers at least workload, environment, owner, and cost center. Azure Policy MUST enforce or append these tags.
- Tag names SHALL use consistent lowercase keys and a controlled vocabulary of values, because tag values are case-sensitive. Tags SHALL NOT hold secrets or sensitive data.

Compliance, audit and exceptions

- The organization MUST map each applicable regulatory or contractual obligation, such as HIPAA, PCI-DSS, or SOC 2, to specific Azure Policy initiatives or controls, and MUST review the mapping at least annually.

- Compliance status for every production subscription SHALL be evaluated continuously through Azure Policy compliance reporting and Microsoft Defender for Cloud. The results MUST be retained as audit evidence.
- Azure Activity logs for every subscription MUST be exported to a central Log Analytics workspace or storage account with a defined retention period.
- Every exception to a policy or standard MUST be recorded with an owner, a documented justification, a compensating control, and an expiration date no later than twelve months after approval. Expired exceptions MUST be re-approved or removed.
- Designs for new or materially changed production workloads SHALL be reviewed against the Well-Architected Framework by an architecture review function. The review SHOULD be supported by automated policy guardrails rather than manual gating of every change.

Sources

- https://learn.microsoft.com/azure/cloud-adoption-framework/ready/landing-zone/design-area/resource-org-management-groups
- https://learn.microsoft.com/azure/cloud-adoption-framework/ready/landing-zone/design-area/governance
- https://learn.microsoft.com/azure/cloud-adoption-framework/ready/azure-best-practices/resource-naming
- https://learn.microsoft.com/azure/cloud-adoption-framework/ready/azure-best-practices/resource-tagging

Chapter 12 — Disaster Recovery, Business Continuity and Performance Efficiency

Intent

Recovery targets, tested failover, and measured performance determine whether a workload meets its commitments when conditions are bad. Untested backups and untested scaling assumptions fail at the worst moment. This chapter sets obligations for recovery objectives, multi-region design, restore and failover testing, load testing, capacity planning, and caching.

Requirements

Recovery objectives and planning

- Every production workload SHALL be assigned a business criticality tier, and every tier MUST have documented RTO and RPO targets. Components with different criticality inside one workload MUST each have their own recovery approach documented.
- Every production workload MUST have a written disaster recovery plan covering all components and the system as a whole, with explicit activation criteria that distinguish a disaster from a minor incident.
- The disaster recovery plan MUST name owners for declaring a disaster, executing recovery, communicating with stakeholders, and closing the incident, and it SHALL define escalation paths and prepared communication channels.
- The disaster recovery plan MUST be reviewed and updated at least annually and after every architecture change that affects recovery.

Multi-region design and data replication

- Each workload's design document MUST state the recovery pattern chosen, such as active-active, active-passive warm standby, or active-passive cold standby, and show that the pattern can meet the documented RTO.
- The secondary region of every multi-region production workload MUST have its network topology, identity configuration, and deployment automation prepared before a disaster, so that recovery does not depend on building them during the event.
- Data replication mode SHALL follow RPO targets, with synchronous replication used where zero data loss is required and asynchronous replication accepted only where the RPO tolerates the lag. Replication lag MUST be monitored and alerted.
- Workloads SHOULD be isolated into components that can be failed over independently, and each partial-failover path MUST be documented and tested separately.
- Failback to the primary region MUST be planned and documented as a separate procedure from failover.

- Traffic redirection between regions SHALL be automated through a global routing service, such as Azure Front Door or Azure Traffic Manager, for workloads whose RTO is shorter than the time a manual DNS change would take.

Backup and restore

- Every stateful production data store MUST have automated backups, with a retention period aligned to the RPO and to regulatory retention requirements, and at least one copy stored in a different region or fault domain than the primary.
- Backups of production data MUST be protected from deletion and tampering through immutability or soft-delete and separate access controls.
- Restore procedures for every production data store SHALL be tested at least annually and after any change to backup configuration or data schema. Test results MUST include measured restore time compared with the RTO and a completeness check of restored data.
- Restore tests SHALL be performed in an environment separate from production.

Failover drills and reliability testing

- A full or partial failover drill for every mission-critical and business-critical workload MUST be run at least annually. The drill MUST measure total recovery time across the whole critical flow, including detection time, against the documented RTO and RPO.
- Disaster recovery drills MUST produce a recorded after-action report, and identified gaps SHALL be tracked to closure.
- Fault injection or chaos testing SHOULD be run against production-like environments, and its scope MUST stay within the workload's error budget.

Load testing and performance targets

- Every production workload MUST define measurable performance targets, such as latency at a stated percentile, throughput, and concurrent users, derived from its SLOs and documented with per-layer performance budgets.
- Every performance test MUST have pass and fail acceptance thresholds for each target metric, so that results are unambiguous.
- Load tests SHALL be executed in the release pipeline for every workload release, and the results MUST be compared against a stored performance baseline to detect regression.
- Load tests for mission-critical workloads MUST run in an environment that matches production in compute SKUs, autoscale settings, caching configuration, and dependencies.
- Production load testing, where performed, MUST be limited to controlled windows with progressive traffic increases, stated safeguards, and a rollback plan.
- Performance results MUST be re-tested at least quarterly against the baseline for every production workload to detect gradual drift.

Capacity planning

- Capacity planning MUST be performed before any predicted change in usage, such as seasonal peaks, product launches, or marketing events. It SHALL be based on historical utilization data, or on pilots and expert estimates when no history exists.
- The capacity plan for every production workload MUST be reviewed at least annually and MUST include a demand forecast, identified bottleneck components, and the resource quotas and regional capacity that need to be reserved or increased.
- Subscription quotas and regional capacity limits for every service that a critical workload depends on MUST be verified in both primary and secondary regions before launch.

Caching

- Cache use SHALL be limited to data that is read frequently and changed infrequently. A cache MUST NOT be the authoritative store of critical data, and the system of record MUST remain a persistent data store.
- Every cached item MUST have an explicit expiration policy, and the design MUST describe how stale or inconsistent cached data is detected or invalidated.
- Applications MUST continue to operate, with degraded performance if necessary, when the cache is unavailable. Cache seeding of a large cache SHOULD be staged to avoid overloading the data store at startup.
- Workloads with multiple application instances that need a consistent view of cached data SHOULD use a shared cache, such as Azure Managed Redis, instead of per-instance in-memory caches.

Sources

- https://learn.microsoft.com/azure/well-architected/reliability/disaster-recovery
- https://learn.microsoft.com/azure/well-architected/reliability/testing-strategy
- https://learn.microsoft.com/azure/well-architected/performance-efficiency/performance-test
- https://learn.microsoft.com/azure/well-architected/performance-efficiency/capacity-planning
- https://learn.microsoft.com/azure/architecture/best-practices/caching

Chapter 13 — Multitenancy and SaaS Architecture

Intent

A multitenant solution serves several customers from shared components, so a weakness in isolation, fairness or tenant identification affects every customer at once. This chapter sets obligations for defining tenants, choosing tenancy models, keeping tenant data separate, controlling noisy neighbours, and automating the tenant lifecycle. Following it lets a provider scale tenants without trading away security, reliability or cost transparency.

Requirements

Tenant definition and tenancy model

- The solution architecture document MUST define what a tenant is for the solution (for example a customer organization, a customer division, or a consumer group) and MUST state how tenants differ from individual users. An unclear tenant definition leads to inconsistent isolation and billing boundaries.
- The architecture document MUST state the expected number of tenants at launch and the planned growth, and the tenancy model MUST be justified against that projection.
- The architecture document MUST record, for each solution component (compute, data, messaging, identity), the selected tenancy model (shared, sharded or dedicated per tenant) together with the isolation, cost and compliance reasons for the choice.
- The architecture document SHALL document the tenancy-model decision as an architecture decision record that lists business objectives, compliance needs, scale, automation capacity and SLA expectations as decision inputs.
- Where a design deploys dedicated Azure resources per tenant, the design SHOULD place each tenant in its own Azure subscription or SHALL document how subscription quotas and limits are managed as the tenant count grows.
- The architecture document MUST distinguish logical tenants from physical deployments when more than one tenant can share a deployment, and MUST describe how tenants are mapped to deployments.

Tenant isolation and data protection

- The design MUST include a documented tenant isolation model that identifies, for every shared component, how one tenant is prevented from reading or modifying another tenant's data.
- Application code and data-access layers MUST apply the tenant identifier as a mandatory filter or partition key on every query against shared data stores. Row-level security, per-tenant schemas, per-tenant databases or equivalent controls SHALL back the application-level filter where the data store supports them.
- The design MUST NOT allow any request to reach tenant data without a resolved and authorized tenant context.

- The design MUST test tenant isolation continuously, including automated cross-tenant access tests in the release pipeline for every production release.
- Tenants with stricter isolation or compliance needs SHOULD be offered a dedicated deployment or dedicated data store, and the document MUST describe how those tenants are routed to it.
- The design MUST document the data residency location of each tenant's data and SHALL ensure that tenant data, backups and replicas remain within the region or geography the tenant contracted for.
- The solution MUST support per-tenant encryption key options (for example customer-managed keys in Azure Key Vault) wherever tenants have contractual requirements for key control, and the design SHALL document which tiers receive them.
- Custom domain handling MUST include a process for removing DNS records when a tenant is offboarded, so that dangling DNS entries cannot be used for subdomain takeover.

Tenant-aware identity and request mapping

- The identity design MUST map every authenticated request to exactly one tenant by using a verifiable claim in the token, a trusted lookup, or another documented mechanism, and MUST NOT derive the tenant from an unvalidated client-supplied value.
- The architecture document MUST state how tenant context is propagated across services, queues and background jobs, so that downstream components enforce the same tenant boundary.
- The design SHALL apply least privilege and zero trust principles to tenant administrators, and tenant-level roles MUST NOT grant access to platform-level management operations.
- Operator and support access to tenant data MUST be limited to approved, time-bound and logged sessions.

Noisy neighbour and fairness

- The design MUST include resource governance that prevents a single tenant from exhausting shared capacity, using per-tenant quotas, rate limits or throttling that return a defined error to the caller.
- The platform MUST monitor resource consumption per tenant as well as in aggregate, and SHALL alert on per-tenant usage spikes before shared capacity is exhausted.
- The design SHOULD offer higher-isolation tiers (reserved throughput, premium service tiers or dedicated instances) for tenants whose workload would otherwise threaten shared capacity.
- The team MUST define service-level objectives for the solution based on tenant requirements and SHALL load-test the solution at the projected maximum tenant count and at the projected peak load of its largest tenant.

Tenant lifecycle, metering and operations

- The platform MUST automate tenant onboarding, configuration, upgrade and offboarding by using infrastructure as code or equivalent scripted workflows, and manual per-tenant provisioning steps SHALL NOT exist for production tenants.
- The offboarding process MUST delete or return tenant data within a documented retention period and MUST record the completion of deletion for audit.
- The solution MUST measure consumption per tenant and SHALL correlate it with infrastructure cost, so that the pricing model reflects actual resource use.
- The design SHOULD separate the control plane (tenant management, provisioning) from the data plane (tenant workload traffic) and MUST document their different scaling and availability requirements.
- The release process MUST define how service updates are rolled out across tenants, including whether tenants can be assigned to update rings or deferred, and SHALL support rollback per ring.

Sources

- https://learn.microsoft.com/azure/architecture/guide/multitenant/overview
- https://learn.microsoft.com/azure/architecture/guide/multitenant/considerations/tenancy-models
- https://learn.microsoft.com/azure/architecture/guide/multitenant/checklist
- https://learn.microsoft.com/azure/architecture/antipatterns/noisy-neighbor/noisy-neighbor

Chapter 14 — AI and Machine Learning Workloads

Intent

AI workloads add risks that conventional applications lack, such as unsafe or ungrounded output, prompt injection, model drift, model retirement and unbounded token spend. This chapter requires that safety, data governance, identity, evaluation and cost controls are designed in and verified rather than assumed from the hosting platform. It applies to both classic machine learning and generative AI workloads.

Requirements

Architecture and layering

- The AI workload design MUST define separate layers for client, intelligence (routing, orchestration, agents), inferencing, knowledge (grounding data) and tools, and each layer SHALL enforce its own identity, authorization and caching policy.
- The client layer MUST remain thin and SHALL NOT contain model invocation logic, prompt content, or credentials for model endpoints.
- The design MUST route model calls from clients through a back-end gateway (for example Azure API Management or an equivalent AI gateway) that enforces authentication, quotas, safety filtering, routing and logging.
- The design MUST abstract models and tools behind stable interfaces so that a model or provider can be replaced without changes to application code.
- The architecture document MUST define measurable business success metrics and regulatory constraints for the AI capability before the model and platform are selected.

Responsible AI and content safety

- Every AI workload that produces user-facing content MUST apply input and output content filtering (for example Azure AI Content Safety or platform-provided filters), and the team SHALL verify the configured filter levels by testing instead of assuming defaults.
- The design MUST include defenses against prompt injection and jailbreak attempts, including prompt shielding or equivalent controls for both user input and retrieved documents.
- The workload MUST set limits on prompt length, output tokens and request rate per identity, so that abuse cannot cause unbounded consumption.
- The team MUST complete and record a responsible AI impact assessment, covering harms, affected groups and mitigations, before any AI feature enters production.
- AI features that influence decisions about people SHALL provide human oversight, and the design MUST document when a human reviews or can override the output.
- The design MUST disclose to users when content is AI-generated and SHOULD provide a feedback channel for reporting harmful or incorrect output.

Model, prompt and data governance

- The team MUST record the source, version, licence terms and intended use of every model in a model inventory and MUST confirm that the licence permits the intended commercial use.
- The workload MUST keep prompts, system messages and model configuration in version control and SHALL deploy them through a controlled release process, so that telemetry can be correlated with a specific prompt version.
- The design MUST plan for model deprecation by recording the retirement date of each hosted model version and by providing a tested fallback model or migration path before that date.
- Training, fine-tuning and grounding data MUST have a documented owner, classification, lineage and retention period, and personal or confidential data SHALL NOT be used for training unless the legal basis is recorded.
- Retrieval over grounding data MUST enforce the end user's authorization, by passing user or tenant identity to the retrieval layer so results are security-trimmed, and AI components MUST NOT read data stores directly.

Evaluation and monitoring

- The team MUST evaluate each model and prompt change against a versioned test dataset for groundedness, relevance, safety and accuracy before release, and SHALL block release when results fall below defined thresholds.
- The workload MUST include red-team or adversarial testing of the AI feature before initial production release and after any change of model, grounding source or system prompt.
- The production workload MUST monitor model quality indicators (accuracy drift, inference latency, confidence scores and input data distribution) and SHALL alert when a defined threshold is breached.
- The workload MUST log prompts, completions and tool calls with user identity and prompt version for audit, and the log retention and redaction rules SHALL be documented for any personal data they contain.

Identity, secrets and network

- Services calling Azure OpenAI, Azure AI services or Azure Machine Learning endpoints MUST authenticate with managed identities or Microsoft Entra ID, and API keys MUST NOT be embedded in code or configuration files.
- Where API keys cannot be avoided, the workload MUST store them in Azure Key Vault and SHALL rotate them at least every 90 days.
- Production AI endpoints and grounding data stores SHOULD be reachable only through private endpoints, and public network access MUST be disabled unless a documented exception exists.
- Agents and tools MUST run with the least-privilege identity needed for their actions, and any tool that changes state SHALL require explicit authorization checks independent of the model's output.

Cost and capacity

- The workload MUST define token and compute budgets and quotas per environment, per application and per tenant or user where applicable, and SHALL alert when consumption reaches defined percentages of the budget.
- The design SHOULD use caching, model routing to smaller models for simple tasks, and batch processing where latency allows, and MUST document capacity choices (provisioned throughput or pay-as-you-go) with their rationale.

Sources

- https://learn.microsoft.com/azure/well-architected/ai/application-design
- https://learn.microsoft.com/azure/well-architected/security/secure-development-lifecycle

Chapter 15 — Sustainability and Efficient Resource Use

Intent

Every design decision affects how much compute, storage and network a workload consumes, and idle or oversized resources waste energy and money alike. Sustainability depends on removing waste, measuring emissions and assigning ownership, without undermining reliability, security or performance. This chapter turns those practices into obligations for design and operations.

Requirements

Governance and measurement

- Every production workload MUST have a named owner accountable for resource efficiency and sustainability targets, and the owner SHALL be recorded in the workload documentation.
- The workload team MUST define at least one measurable efficiency or emissions target (for example average CPU utilization, carbon per transaction or monthly emissions) and SHALL review it at least quarterly.
- The team SHOULD measure workload emissions by using Azure carbon optimization or an equivalent tool, and MUST record the baseline and each reported change.
- The architecture document MUST record the sustainability trade-offs of reliability, performance and security decisions, such as multiregion duplication, extra caching or long telemetry retention.

Right-sizing and utilization

- The team MUST right-size every production compute resource against measured utilization at least every six months, and SHALL document the observed utilization used as evidence.
- The design MUST define a target average utilization for each compute tier and SHALL justify any tier running persistently below that target.
- Stateless and variable-load components MUST use autoscaling (for example virtual machine scale sets, Azure Kubernetes Service autoscalers or App Service autoscale) with a defined minimum and maximum instance count, instead of being sized for peak demand.
- Non-production environments MUST shut down or scale to zero outside documented working hours, unless the document records a reason for continuous operation.
- The team MUST identify and remove idle or orphaned resources (unattached disks, unused public IP addresses, empty clusters and unused snapshots) at least monthly, and SHOULD automate the detection.
- The design SHOULD prefer managed PaaS or serverless services over self-managed virtual machines where they meet requirements, because shared platform capacity is used more efficiently.

Region and placement

- The design MUST document the selection of each deployment region and SHALL weigh carbon intensity, latency, data residency and service availability.
- Batch, training and other delay-tolerant workloads SHOULD be scheduled in lower-carbon regions or time windows when no latency or residency constraint prevents it.
- The design MUST NOT replicate workloads across more regions than the documented availability objective requires, and every extra region SHALL be justified by a reliability or residency requirement.
- Components that exchange large volumes of data SHOULD be co-located in the same region and zone to avoid unnecessary data transfer.

Efficient application and data design

- The design SHOULD use caching, asynchronous and event-driven processing, and efficient data access patterns to reduce repeated computation, and the architecture document MUST identify the main hot paths for which this was considered.
- The workload MUST set a documented retention period for every data store, log and backup, and automated lifecycle rules SHALL delete or archive data when that period ends.
- Data that is rarely accessed MUST be moved to cooler or archive storage tiers by lifecycle policy rather than kept in the hot tier.
- The workload MUST NOT replicate data to additional copies, regions or tiers beyond what its recovery objectives require.
- Telemetry collection MUST be limited to data with a defined operational purpose, with log levels, sampling and retention set per environment, and verbose logging SHALL NOT be enabled by default in production.
- Container images and build artifacts MUST be retained under a defined cleanup policy, and the CI/CD pipeline SHOULD avoid redundant full builds and test runs.

AI and data-intensive workloads

- AI workloads SHOULD use the smallest model and the lowest-precision compute that satisfy the documented quality target, and the selection rationale MUST be recorded.
- GPU and other accelerator resources MUST be released or scaled down when training or inference jobs complete, and idle accelerator time SHALL be reported at least monthly.

Sources

- https://learn.microsoft.com/azure/well-architected/sustainability/overview
- https://learn.microsoft.com/azure/well-architected/sustainability/sustainability-design-principles
- https://learn.microsoft.com/azure/well-architected/ai/application-design

Chapter 16 — Architecture Documentation, Decisions and Review

Intent

A workload's architecture is the sum of its decisions, and decisions that are not recorded are forgotten, re-debated or silently contradicted. Documentation that is owned, versioned and reviewed gives operators, auditors and incident responders a single source of truth. This chapter sets obligations for decision records, diagrams, inventories, threat models, reviews and risk acceptance.

Requirements

Architecture decision records

- Every workload MUST maintain an architecture decision record (ADR) log that is started at the beginning of the workload and kept for its entire lifetime.
- The ADR log MUST contain a record for each architecturally significant decision, meaning each decision that affects structure, a key quality attribute, or that is hard to reverse.
- Every ADR MUST follow one common template containing at least the problem context, the options considered, the decision outcome, the trade-offs accepted, and the status.
- Every ADR MUST carry a status of Proposed, Accepted or Superseded, and SHOULD record the confidence level of the decision when confidence is low.
- The ADR log SHALL be append-only, so that an accepted record MUST NOT be edited to change its decision, and a changed decision MUST be recorded as a new ADR that links to the record it supersedes.
- An ADR that spans several phases (short-term, mid-term, long-term) MUST be split into one record per phase.
- Brownfield workloads without an ADR log MUST create one and SHOULD reconstruct records for known past decisions.

Diagrams and workload documentation

- The workload documentation MUST include a context diagram, a logical or component diagram, and a deployment or network topology diagram, each showing the trust boundaries.
- Each diagram MUST identify its subject, its version or date, and its owner, and every diagram SHALL use a consistent notation with a legend.
- The documentation MUST describe the data flows for each category of sensitive data, including where data is stored, processed and transmitted.
- The documentation MUST list the workload's reliability targets (SLOs, RTO and RPO), and SHALL link each to the design elements that support it.

- The documentation MUST include operational runbooks for deployment, rollback, failover and incident response, and the runbooks SHALL be tested at least annually.

Repository, ownership and versioning

- All workload documentation MUST be stored in a single repository that is accessible to the workload team, auditors and incident responders, and SHALL be referenced as the source of truth.
- Architecture documents MUST be stored under version control with change history, and each released version SHALL be identified by a version number or tag.
- Every architecture document MUST name an accountable owner and a review date, and a document without a current owner SHALL be treated as out of date.
- The workload team MUST update the affected documents in the same change that modifies the architecture, and the change review SHALL verify that the update occurred.

Dependency inventory

- The workload MUST maintain an inventory of all internal and external dependencies, including cloud services, third-party APIs, libraries, container base images and data sources.
- Each inventory entry MUST record the dependency's owner or vendor, version, criticality, and the effect on the workload if it is unavailable.
- The dependency inventory MUST be reviewed at least quarterly, and a dependency approaching end of support SHALL have a replacement or upgrade plan recorded.

Threat modeling

- Every workload MUST have a documented threat model, created during design, that identifies assets, trust boundaries, threats and mitigations, for example by using STRIDE.
- The threat model MUST be reviewed and updated at least annually and whenever the architecture, data classification or an external integration changes.
- Each identified threat MUST map to a mitigation, an owner and a status, and the status SHALL be one of mitigated, accepted or open.

Review cadence and risk acceptance

- Every production workload MUST undergo a documented architecture review (for example Azure Well-Architected Review) at least annually and before any major redesign.
- Each review MUST record its findings, the assigned owner and due date for every finding, and the results SHALL be tracked until closed.
- Any deviation from a mandatory policy requirement MUST be recorded as an exception that states the requirement, the justification, compensating controls, the approver and an expiry date.
- An exception MUST have a named business-level approver with the authority to accept the risk, and MUST NOT remain active beyond its expiry date unless it is formally renewed.

- Risk acceptance for security trade-offs MUST be documented and signed by accountable stakeholders when a recommended control is not funded or implemented.
- The exception register MUST be reviewed at least quarterly, and expired entries SHALL be closed or renewed.

Sources

- https://learn.microsoft.com/azure/well-architected/architect-role/architecture-decision-record
- https://learn.microsoft.com/azure/well-architected/security/secure-development-lifecycle